

INSTITUT SUPÉRIEUR D'INFORMATIQUE ET MULTIMÉDIA
DE GABÈS

Option : Systèmes Embarqués & IOT

Sécurité des Systèmes Embarqués

RAPPORT DE TRAVAUX PRATIQUES

Cryptage, Hachage & Codes Correcteurs

Simulation d'attaques MitM, Intégrité & Validation IoT

Réalisé par :

Gouider Mohamed

Année universitaire : 2024 – 2025

Table des matières

1	Introduction	1
2	Algorithme AES (128 bits)	1
2.1	Vue d'ensemble	1
2.2	Implémentation	1
2.2.1	Multiplication dans $GF(2^8)$ — <code>gf_mul</code>	1
2.2.2	Expansion de clé — <code>aes_key_expansion</code>	1
2.2.3	Transformations par tour	2
2.2.4	Conversion état — <code>bytes_to_state</code> / <code>state_to_bytes</code>	2
2.3	Chiffrement — <code>aes_encrypt</code>	3
2.4	Déchiffrement — <code>aes_decrypt</code>	3
2.5	Tests et validation	3
3	Algorithme DES (64 bits)	4
3.1	Vue d'ensemble	4
3.2	Génération des sous-clés — <code>des_generate_subkeys</code>	4
3.2.1	Permutation — <code>permute</code>	4
3.2.2	Décalage circulaire gauche — <code>left_shift_28</code>	4
3.3	Fonction de Feistel — <code>feistel_f</code>	4
3.4	Chiffrement et déchiffrement — <code>des_process</code>	5
3.5	Tests et validation	5
4	Algorithme de hachage MD5	5
4.1	Vue d'ensemble	5
4.2	Implémentation	5
4.2.1	Padding et initialisation	5
4.2.2	Traitement par blocs	6
4.2.3	Finalisation	6
4.3	Tests et validation	6
5	Codes de détection et correction d'erreurs	7
5.1	Contrôle de redondance cyclique (CRC)	7
5.1.1	Implémentation	7
5.1.2	Validation par simulation d'erreur	7
5.2	Code de Hamming	7
5.3	Code par répétition	8
6	Dockerisation	8
6.1	Architecture	8
6.2	Dockerfile et docker-compose	8
6.3	Résultats d'exécution	9
7	Intégration IoT : ESP32 & Serveur AES-128	9
7.1	Architecture et protocole de défi	9
7.2	Résultats et validation	9
8	Conclusion	9

Table des figures

2.1	Fonction <code>gf_mul</code> – multiplication dans $GF(2^8)$	1
2.2	Fonction <code>aes_key_expansion</code> – expansion de la clé AES-128	1
2.3	Fonction <code>add_round_key</code>	2
2.4	Fonction <code>sub_bytes</code> – substitution via la S-Box	2
2.5	Fonction <code>shift_rows</code>	2
2.6	Fonction <code>mix_columns</code>	2
2.7	Fonctions <code>bytes_to_state</code> et <code>state_to_bytes</code>	2
2.8	Fonction <code>aes_encrypt</code> – chiffrement AES-128	3
2.9	Fonction <code>aes_decrypt</code> – déchiffrement AES-128	3
2.10	Résultat des tests AES dans <code>main.c</code>	3
3.1	Fonction <code>des_generate_subkeys</code> – génération des 16 sous-clés	4
3.2	Fonction <code>permute</code> – permutation générique de bits	4
3.3	Fonction <code>left_shift_28</code> – décalage circulaire gauche	4
3.4	Fonction <code>feistel_f</code> – fonction de Feistel	4
3.5	Fonction <code>des_process</code> – chiffrement et déchiffrement DES	5
3.6	Résultat des tests DES dans <code>main.c</code>	5
4.1	Signature de la fonction d’initialisation <code>md5_init</code>	6
4.2	Signature de la fonction globale de traitement <code>md5_digest</code>	6
4.3	Signature de la fonction interne de compression <code>md5_block</code>	6
4.4	Primitives logiques et déclaration de la macro <code>STEP</code>	6
4.5	Signature de la fonction d’extraction d’empreinte <code>md5_output</code>	6
4.6	Suite de validation et point d’entrée <code>main</code> dans <code>md5_test.c</code>	6
5.1	Exécution du programme de test du module CRC-8	7
5.2	Calcul du syndrome et correction d’erreur par le code de Hamming	8
5.3	Simulation du canal bruité et correction par vote majoritaire	8

1 Introduction

À l'ère du numérique, la sécurité de l'information est devenue un enjeu critique. Des milliards d'appareils échangent chaque seconde des données sensibles : transactions bancaires, communications médicales, commandes industrielles. Garantir leur **confidentialité**, **intégrité** et **authenticité** est indispensable, en particulier dans les systèmes embarqués où les ressources sont contraintes.

Ce rapport couvre l'implémentation des mécanismes suivants :

- **AES** — standard mondial de chiffrement symétrique (HTTPS, VPN, WPA2).
- **DES** — précurseur d'AES, fondement des réseaux de Feistel.
- **MD5** — hachage pour la vérification d'intégrité des données.
- **CRC** — détection d'erreurs (Ethernet, USB).
- **Hamming** — correction d'un bit erroné (mémoires ECC).
- **Répétition** — redondance par vote majoritaire.

L'ensemble est conteneurisé via **Docker** pour garantir la reproductibilité des tests, et validé sur une architecture IoT réelle avec une carte **ESP32**.

2 Algorithme AES (128 bits)

2.1 Vue d'ensemble

L'**Advanced Encryption Standard (AES)** est un algorithme de chiffrement symétrique par blocs standardisé par le NIST sous la référence FIPS-197. Ce TP utilise la variante **AES-128** : blocs de 128 bits, clé de 128 bits et **10 tours** de chiffrement. L'état interne est une matrice 4×4 d'octets manipulée colonne par colonne. Les constantes (S-Box, Inverse S-Box, RCON) sont définies dans `aes.h`.

2.2 Implémentation

2.2.1 Multiplication dans $GF(2^8)$ — `gf_mul`

Toutes les opérations arithmétiques de l'AES s'effectuent dans le corps fini $GF(2^8)$, modulo le polynôme irréductible $x^8 + x^4 + x^3 + x + 1$ (0x11B). La fonction `gf_mul(a, b)` implémente cette multiplication bit à bit : si le bit de poids faible de `b` vaut 1, on XOR le résultat avec `a` ; on décale `a` à gauche et on applique la réduction modulaire si nécessaire.

```
uint8_t gf_mul(uint8_t a, uint8_t b);
```

FIGURE 2.1 – Fonction `gf_mul` – multiplication dans $GF(2^8)$

2.2.2 Expansion de clé — `aes_key_expansion`

Génère les **11 sous-clés** de 16 octets à partir de la clé de 128 bits. À chaque tour r , le dernier mot subit *RotWord*, *SubWord* (via S-Box), puis un XOR avec `RCON[r]`. Les mots suivants sont calculés par XOR en chaîne.

```
void aes_key_expansion(const uint8_t key[16], uint8_t round_keys[11][16]);
```

FIGURE 2.2 – Fonction `aes_key_expansion` – expansion de la clé AES-128

2.2.3 Transformations par tour

Chaque tour AES applique quatre transformations successives :

add_round_key XOR octet par octet entre l'état et la sous-clé du tour. C'est la seule opération qui mêle directement la clé aux données.

```
void add_round_key(uint8_t state[4][4], const uint8_t rkey[16]);
```

FIGURE 2.3 – Fonction `add_round_key`

sub_bytes Substitue chaque octet via la S-Box (ou l'Inverse S-Box en déchiffrement).

```
void sub_bytes(uint8_t state[4][4], const uint8_t sbbox[256]) ;
```

FIGURE 2.4 – Fonction `sub_bytes` – substitution via la S-Box

shift_rows Rotation cyclique des lignes : la ligne r est décalée de r positions (gauche en chiffrement, droite en déchiffrement), contrôlée par le paramètre `inv`.

```
void shift_rows(uint8_t state[4][4], int inv);
```

FIGURE 2.5 – Fonction `shift_rows`

mix_columns Transformation linéaire sur chaque colonne dans $\text{GF}(2^8)$, avec les coefficients $\{2, 3, 1, 1\}$ (chiffrement) ou $\{0x0E, 0x0B, 0x0D, 0x09\}$ (déchiffrement). Le dernier tour de chiffrement omet cette étape.

```
void mix_columns(uint8_t s[4][4], int inv) ;
```

FIGURE 2.6 – Fonction `mix_columns`

2.2.4 Conversion état — `bytes_to_state` / `state_to_bytes`

Conversion entre le flux linéaire de 16 octets et la matrice 4×4 , en respectant l'ordre colonne-majeur de FIPS-197 : l'octet i se place en $(i \bmod 4, i \div 4)$.

```
/* Copie linéaire in[16] → état 4x4 colonne-majeur */
static void bytes_to_state(const uint8_t in[16], uint8_t state[4][4]) {
    for (int i = 0; i < 16; i++) state[i % 4][i / 4] = in[i];
}

/* Copie état 4x4 → out[16] colonne-majeur */
static void state_to_bytes(const uint8_t state[4][4], uint8_t out[16]) {
    for (int i = 0; i < 16; i++) out[i] = state[i % 4][i / 4];
}
```

FIGURE 2.7 – Fonctions `bytes_to_state` et `state_to_bytes`

2.3 Chiffrement — aes_encrypt

Orchestre les 10 tours de l’AES-128 :

1. Expansion de la clé (aes_key_expansion).
2. Conversion en état 4×4 (bytes_to_state).
3. add_round_key avec la sous-clé du tour 0.
4. Tours 1–9 : sub_bytes → shift_rows → mix_columns → add_round_key.
5. Tour 10 : sub_bytes → shift_rows → add_round_key (sans mix_columns).
6. Conversion de l’état en sortie (state_to_bytes).

```
void aes_encrypt(const uint8_t in[16], const uint8_t key[16], uint8_t out[16]);
```

FIGURE 2.8 – Fonction aes_encrypt – chiffrement AES-128

2.4 Déchiffrement — aes_decrypt

Applique les transformations inverses dans l’ordre inverse : après add_round_key avec la sous-clé du tour 10, les tours 9 à 1 appliquent shift_rows(inv=1) → sub_bytes(INV_SBOX) → add_round_key → mix_columns(inv=1). Le tour final omet mix_columns, symétrique au chiffrement.

```
void aes_decrypt(const uint8_t in[16], const uint8_t key[16], uint8_t out[16]);
```

FIGURE 2.9 – Fonction aes_decrypt – déchiffrement AES-128

2.5 Tests et validation

Validation avec le vecteur officiel de l’annexe B de FIPS-197 :

Paramètre	Valeur
Clé	2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C
Clair	32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34
Chiffré	39 25 84 1D 02 DC 09 FB DC 11 85 97 19 6A 0B 32

La fonction test_aes() vérifie la concordance du chiffré avec la valeur attendue ainsi que le roundtrip (déchiffrement retrouve le clair original).

```
--- [ AES-128 ] ---
Plaintext      : 32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34
Ciphertext     : 39 25 84 1D 02 DC 09 FB DC 11 85 97 19 6A 0B 32
CT attendu    : 39 25 84 1D 02 DC 09 FB DC 11 85 97 19 6A 0B 32
D|®chiffr|®    : 32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34
Vecteur CT    : OK
Roundtrip     : OK
```

FIGURE 2.10 – Résultat des tests AES dans main.c

3 Algorithmes DES (64 bits)

3.1 Vue d'ensemble

Le **Data Encryption Standard (DES)** est un chiffrement symétrique par blocs (FIPS-46-3), opérant sur des blocs de **64 bits** avec une clé de 64 bits dont seulement **56 bits** sont effectifs. Il repose sur un **réseau de Feistel à 16 tours**.

Note : DES est obsolète en production. Son implémentation est strictement pédagogique, conformément à FIPS-46-3.

3.2 Génération des sous-clés — `des_generate_subkeys`

Produit les **16 sous-clés** de 48 bits. La clé subit la permutation PC-1 (56 bits effectifs), puis est divisée en deux demi-clés C et D de 28 bits. À chaque tour : décalage circulaire gauche + compression PC-2 → sous-clé de 48 bits.

```
void des_generate_subkeys(const uint8_t key_bytes[8], uint8_t subkeys[16][48]);
```

FIGURE 3.1 – Fonction `des_generate_subkeys` – génération des 16 sous-clés

3.2.1 Permutation — `permute`

Fonction générique réordonnant les bits d'un vecteur selon une table donnée. Utilisée pour toutes les permutations DES : PC-1, PC-2, IP, FP, EP et PBOX.

```
void permute(const int *table, int n, const uint8_t *in, uint8_t *out);
```

FIGURE 3.2 – Fonction `permute` – permutation générique de bits

3.2.2 Décalage circulaire gauche — `left_shift_28`

Décalage circulaire gauche de `n` positions sur une demi-clé de 28 bits. Le nombre de positions suit le calendrier `SHIFT_SCHEDULE` de FIPS-46-3 (1 ou 2 positions selon le tour).

```
void left_shift_28(uint8_t *half, int n);
```

FIGURE 3.3 – Fonction `left_shift_28` – décalage circulaire gauche

3.3 Fonction de Feistel — `feistel_f`

Prend la demi-droite R (32 bits) et la sous-clé (48 bits), produit 32 bits :

1. **Expansion EP** : R étendu de 32 à 48 bits.
2. **XOR** avec la sous-clé du tour.
3. **Substitution S-Box** : 8 groupes de 6 bits → 4 bits chacun (32 bits total).
Ligne = bits 1 et 6 ; colonne = bits 2 à 5.
4. **Permutation PBOX** : réordonnancement des 32 bits.

```
void feistel_f(const uint8_t R[32], const uint8_t subkey[48], uint8_t result[32]);
```

FIGURE 3.4 – Fonction `feistel_f` – fonction de Feistel

3.4 Chiffrement et déchiffrement — des_process

Gère chiffrement et déchiffrement via un paramètre `decrypt` :

1. Génération des 16 sous-clés.
2. Conversion en bits et permutation initiale IP.
3. Séparation en demi-blocs L et R (32 bits chacun).
4. 16 tours de Feistel : $\text{nouvel } R = L \oplus f(R, K_r)$, nouveau L = ancien R.
5. Ordre des sous-clés contrôlé par `int idx = decrypt ? (15-r) : r`.
6. Recombinaison $R_{16} \parallel L_{16}$ + permutation finale FP.

```
void des_process(const uint8_t input[8], const uint8_t key[8],
                uint8_t output[8], int decrypt);
```

FIGURE 3.5 – Fonction `des_process` – chiffrement et déchiffrement DES

3.5 Tests et validation

Vecteur de référence FIPS-46-3 :

Paramètre	Valeur
Clé	13 34 57 79 9B BC DF F1
Clair	01 23 45 67 89 AB CD EF
Chiffré	85 E8 13 54 0F 0A B4 05

```
--- [ AES-128 ] ---
Plaintext      : 32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34
Ciphertext     : 39 25 84 1D 02 DC 09 FB DC 11 85 97 19 6A 0B 32
CT attendu    : 39 25 84 1D 02 DC 09 FB DC 11 85 97 19 6A 0B 32
D|®chiffr|®    : 32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34
Vecteur CT    : OK
Roundtrip     : OK
```

FIGURE 3.6 – Résultat des tests DES dans `main.c`

4 Algorithme de hachage MD5

4.1 Vue d'ensemble

MD5 (*Message-Digest Algorithm 5*) est une fonction de hachage cryptographique produisant une empreinte de **128 bits** à partir d'un message de taille arbitraire. L'architecture s'appuie rigoureusement sur la RFC 1321, structurant le traitement via un contexte de hachage dédié.

4.2 Implémentation

4.2.1 Padding et initialisation

Les quatre registres d'état (A, B, C, D) reçoivent des valeurs magiques de 32 bits imposées par le standard. Le message est ensuite complété (*padding*) pour que sa longueur en bits soit congrue à $448 \pmod{512}$.

```
void md5_init(md5_context* ctx);
```

FIGURE 4.1 – Signature de la fonction d'initialisation md5_init

```
void md5_digest(md5_context* ctx, void* buffer, size_t size);
```

FIGURE 4.2 – Signature de la fonction globale de traitement md5_digest

4.2.2 Traitement par blocs

La compression opère sur des blocs de **512 bits** (64 octets). La figure 4.3 montre la signature de la fonction interne. La figure 4.4 expose les primitives logiques (F, G, H, I) et la macro **STEP** appliquée lors des 64 étapes réparties en quatre rondes.

```
static void md5_block(md5_context* ctx, const uint8_t m[64]);
```

FIGURE 4.3 – Signature de la fonction interne de compression md5_block

```
#define F(X, Y, Z) (((X) & (Y)) | ((~(X)) & (Z)))
#define G(X, Y, Z) (((X) & (Z)) | (Y & (~(Z))))
#define H(X, Y, Z) ((X) ^ (Y) ^ (Z))
#define I(X, Y, Z) ((Y) ^ ((X) | (~(Z))))

static uint32_t rotl(uint32_t x, int s);

#define STEP(OP, a, b, c, d, k, s, i)
```

FIGURE 4.4 – Primitives logiques et déclaration de la macro STEP

4.2.3 Finalisation

L'état finalisé des registres est transféré vers un tableau de sortie linéaire de **16 octets**, concluant ainsi le cycle de vie du hachage.

```
void md5_digest(md5_context* ctx, void* buffer, size_t size);
```

FIGURE 4.5 – Signature de la fonction d'extraction d'empreinte md5_output

4.3 Tests et validation

Les vecteurs de validation officiels de la RFC 1321 sont utilisés. La comparaison stricte de blocs mémoire confirme la conformité du code aux standards cryptographiques.

```
int main();
```

FIGURE 4.6 – Suite de validation et point d'entrée main dans md5_test.c

5 Codes de détection et correction d'erreurs

5.1 Contrôle de redondance cyclique (CRC)

Le CRC est une technique de détection d'erreurs basée sur la division polynomiale binaire. L'émetteur calcule un reste (checksum) lié à un polynôme générateur standardisé et l'adjoint au message ; le récepteur vérifie par recalcul.

5.1.1 Implémentation

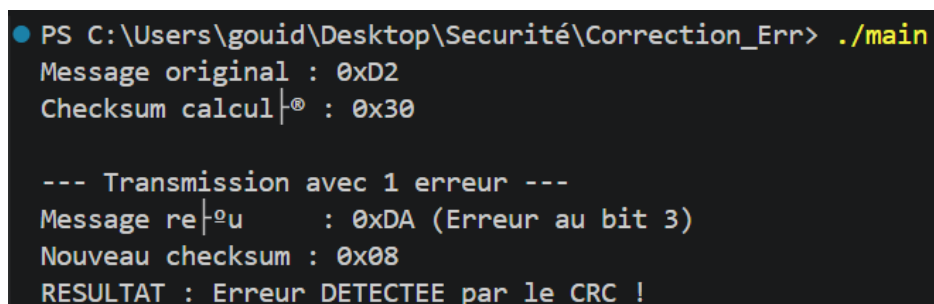
Le polynôme CRC-8 est défini via la macro `POLYNOMIAL`. La routine principale :

```
uint8_t calculer_crc8(uint8_t data);
```

traite les données séquentiellement via des décalages de bits et des XOR.

5.1.2 Validation par simulation d'erreur

Une altération aléatoire sur un bit de la trame est injectée. La figure 5.1 montre l'écart entre le checksum d'origine (0x30) et celui recalculé après altération (0x08), permettant la détection immédiate de l'anomalie.



```
PS C:\Users\gouid\Desktop\Securité\Correction_Err> ./main
Message original : 0xD2
Checksum calculé : 0x30

--- Transmission avec 1 erreur ---
Message reçu : 0xDA (Erreur au bit 3)
Nouveau checksum : 0x08
RESULTAT : Erreur DETECTEE par le CRC !
```

FIGURE 5.1 – Exécution du programme de test du module CRC-8

5.2 Code de Hamming

Le code de Hamming (7,4) insère des bits de parité aux positions stratégiques (puissances de 2) pour détecter *et localiser* précisément un bit corrompu grâce au calcul du syndrome.

Lors de l'injection d'une perturbation à l'indice 0, l'application calcule le syndrome binaire, isole la position exacte du problème et corrige par inversion logique du bit affecté.

```

PS C:\Users\gouid\Desktop\Securité\Correction_Err> ./main_ha
Message Recu : 1010101
--- Simulation : 1 erreur(s) ---
Erreur insérée à l'indice : 0
Message avec erreur : 0010101

Syndrome calculé : 1 (Binaire: 001)
Résultat : Erreur détectée à la position 1.
Action : Bit inversé pour correction.

Message Original : 1 1 0 1
Message Corrigé : 1 1 0 1

```

FIGURE 5.2 – Calcul du syndrome et correction d'erreur par le code de Hamming

5.3 Code par répétition

Chaque bit d'information est transmis **trois fois**. Le décodage applique un *vote majoritaire* pour tolérer l'inversion d'un bit par bloc. Malgré les nombreuses altérations locales simulées, la séquence d'origine est restituée avec succès.

```

PS C:\Users\gouid\Desktop\Securité\Correction_Err> ./main_rep
--- Simulation du Canal (1 erreur par bloc) ---
Bloc 0 : Erreur insérée à l'index local 2
Bloc 1 : Erreur insérée à l'index local 0
Bloc 2 : Erreur insérée à l'index local 2
Bloc 3 : Erreur insérée à l'index local 2
Bloc 4 : Erreur insérée à l'index local 1

--- Résultats ---
Original : 1 0 1 1 0
Reçu (err): 110100110110010
Corrigé : 1 0 1 1 0

Statut : SUCCÈS (Toutes les erreurs ont été corrigées)
PS C:\Users\gouid\Desktop\Securité\Correction_Err>

```

FIGURE 5.3 – Simulation du canal bruité et correction par vote majoritaire

6 Dockerisation

6.1 Architecture

Trois conteneurs isolés sur le réseau virtuel `crypto_net` :

- `emetteur` : initialise un flux binaire contenant la valeur brute `0xD2`.
- `attaquant` : intercepte `trame_saine.bin` et applique $\hat{=}$ `0x01`.
- `recepteur` : récupère `trame_corrompue.bin` (valeur `0xD3`).

6.2 Dockerfile et docker-compose

L'orchestration utilise `python:3.10-slim` pour l'émission et l'attaque, et `gcc:latest` pour le récepteur. L'ensemble est déclenché par `run_demo.bat`.

6.3 Résultats d'exécution

1. **Altération binaire** : passage déterministe de $0xD2$ à $0xD3$ par inversion du bit de poids faible.
2. **Détection** : le binaire `verif_crc` identifie l'anomalie par recalcul avec le polynôme $x^8 + x^2 + x + 1$ ($0x07$).

Pour une visualisation, veuillez consulter le fichier `vid_Docker.mp4`.

7 Intégration IoT : ESP32 & Serveur AES-128

7.1 Architecture et protocole de défi

Un protocole *Challenge-Response* est mis en place entre une carte **ESP32** (client) et un serveur local Python (passerelle d'edge computing).

Flux opérationnel :

1. **Stockage local** : l'ESP32 possède un dictionnaire de trois constantes textuelles associées à leurs trames hexadécimales pré-chiffrées (`main.py`).
2. **Émission du défi** : sélection aléatoire d'une trame chiffrée, transmise via HTTP POST (JSON, port 8080).
3. **Résolution cryptographique** : le serveur appelle `aes_decrypt()` et renvoie le mot en clair.
4. **Validation** : l'ESP32 compare le retour du serveur avec le mot attendu.

7.2 Résultats et validation

Le test confirme la conformité bidirectionnelle entre le code C++ de l'ESP32 et le script Python du serveur. L'absence d'écart algorithmique garantit le déchiffrement exact des blocs de 16 octets à chaque itération. Pour une démonstration visuelle, consulter le fichier `vid_ESP32.mp4`.

8 Conclusion

Ce rapport a présenté l'implémentation complète de six mécanismes fondamentaux de la sécurité informatique : les algorithmes de chiffrement **AES** et **DES**, la fonction de hachage **MD5**, ainsi que les codes de détection et correction d'erreurs **CRC**, **Hamming** et **Répétition**.

Au-delà de la simple implémentation, ce travail a pris soin d'aller plus loin en adoptant deux approches technologiques modernes et particulièrement enrichissantes. D'une part, la **conteneurisation Docker** a permis de simuler une véritable attaque Man-in-the-Middle dans un environnement réseau isolé et entièrement reproductible, transformant un concept théorique abstrait en une démonstration concrète et observable. D'autre part, l'intégration d'une **application IoT réelle** avec la carte ESP32 a permis de valider l'interopérabilité de l'algorithme AES entre des plateformes hétérogènes (C embarqué et Python), dans des conditions proches de la réalité industrielle.

Ces deux expériences illustrent parfaitement une vérité fondamentale de l'apprentissage en informatique : les **nouvelles technologies** — que ce soit la virtualisation, les conteneurs ou les objets connectés — ne sont pas de simples outils supplémentaires, mais de véritables **vecteurs pédagogiques**. Elles rendent les concepts cryptographiques tangibles, les erreurs visibles, et les succès mesurables. C'est précisément en manipulant des systèmes réels, en observant des trames corrompues ou des défis cryptographiques

résolus en temps réel, que la compréhension des bases de la sécurité des systèmes embarqués devient profonde et durable.